Jobblo – Security Fix Log

Fixes for the findings in "Jobblo – Security & Code Review" · last updated 26 Oct 2026 · committed locally in topic commits on branch fixed/secuity, not pushed

Status: All 23 code findings fixed and verified (C2, C3, H1–H9, M1–M12), plus 11 P3 items and the second review round. C1 and C4 are key rotations for the owner. Full backend suite: 1,854 passed, 0 failed. C1 and C4 are key rotations done outside the code.

Every fix ships with an automated test that fails on the old code and passes on the new code, plus live checks against a local copy of the app. The full backend suite passes after each fix.

Progress

ID	Finding	Status	Proof
P0 C1	Azure secret in git history	**Owner action**	Rotate in Entra ID, review sign-ins
P0 C2	WebSocket accepted any website	**Fixed**	4 tests · 3 live checks
P0 C3	Private profile data visible to other users	**Fixed**	8 tests · 6 live checks
P0 C4	Production keys shared outside the server	**Owner action**	Redis rotated; others in progress
P1 H3	Search input used as raw regex; public filter endpoint	**Fixed**	tests · 6 live checks · endpoint removed (404)
P1 H5	Seed scripts could wipe the production database	**Fixed**	9 tests (incl. real scripts)
P1 H1	Mobile: crafted login link switched the signed-in account	**Fixed**	8 checks · typecheck
P1 H2	Mobile: logout left the push token active	**Fixed**	API replay: old order 401/active, new 200/inactive
P1 H4	Two ratings per order	**Fixed**	2 new tests (fail on old code)
P1 H9	2FA set-up possible with only a stolen password	**Fixed**	7 tests (fail on old code) · e2e · 2 live checks
P1 H8	Backend libraries with known security bugs	**Fixed**	npm audit: 3 high → 0 · suite + live checks
P1 H6	Paid money could get stuck after approval or a failed transfer	**Fixed**	13 new tests (8 fail on old code) · full suite
P1 H7	Disputes could freeze money or make the platform pay twice	**Fixed**	11 new tests (all fail on old code)

P2 M1	Error responses leaked internal messages	**Fixed**	9 tests incl. a regression scan of all controllers
P2 M12	Personal data in logs	**Fixed**	redaction tests · app typecheck
P2 M10	Switching the Stripe mode dropped the other mode's webhooks	**Fixed**	6 new tests with real signatures
P2 M2	Pagination unbounded on list endpoints	**Fixed**	8 tests + earlier helper changes
P2 M6	Smaller authorization gaps	**Fixed**	8 tests (14 of 15 in the file fail on old code)
P2 M9	Mobile app: token storage, drafts, double pay, store rules	**Fixed**	4 checks pass · typecheck 0 errors
P2 M5	Sessions and sockets (+ chat parts of M4, M6, M7, M2)	**Fixed**	20+ tests fail on old code
P2 M7	Abuse and cost (AI, error ingest, views)	**Fixed**	8 tests
P2 M3	Upload checks too weak	**Fixed**	13 new tests
P2 M4	Features that silently did not work	**Fixed**	tests in upload, chat and SafePay suites
P2 M11	Deploy and CI	**Fixed**	dry runs, YAML parsed; watch the first real deploy
P3	11 backend low issues (accounts, money, config)	**Fixed**	tests for each; 79 tests in 8 suites re-run
P2 M8	Web app bugs	**Fixed**	8 browser tests + node test, typecheck: no new errors
R2 Round 2	New findings from the second review	**Fixed**	see the R2 sections

Fixes in detail

P0 C2 · WebSocket refused from other websites

Problem: browsers do not apply CORS to WebSockets, so a page on any site could open a Socket.IO connection with a logged-in visitor's cookie and read their notifications and chats.

Fix: the Socket.IO server now refuses the handshake when the `Origin` is not in Jobblo's allowlist (`allowRequest`, reusing the same check as HTTP CORS). Requests without an Origin (mobile app, server) still connect.

Files: `backend/bootstrap/socket.js` · new test `backend/__tests__/socketOrigin.test.js`

Verified:

- Test: foreign origin and lookalike origin refused (403); own origin and no origin accepted. Fails on the old code (2 of 4), passes on the new.
- Live: `https://evil.example` refused · web app `localhost:5173` connects · mobile (no Origin) connects.

P0 C3 · Private profile data no longer visible to other users

Problem: the "public" user projection was the owner's list minus three fields, so any logged-in user got other users' home address, earnings, last login, birth date, Google and Stripe ids; `/users/top` and search returned everyone's email, search matched on email, public lists showed members' emails, and `completed-details` showed both parties' emails before payment.

Fix:

- New explicit allow-list `PUBLIC_FIELDS` (name, avatar, bio, skills, rating, city, portfolio, response stats…). The response cleaner keeps only these plus computed stats, so a query that selects more still cannot leak it.
- Search no longer selects, returns or matches on email.
- Public lists: emails only for the list's own members.
- Order completed-details: emails only once the order is paid (same rule as the order list).
- Search also returns only public jobs (allow-list instead of a blacklist) and caps `page` / `limit` (max 50).

Files: `backend/utils/userProjections.js`, `backend/controllers/users/searchController.js`, `backend/controllers/listController.js`, `backend/controllers/orderController.js` · new test `backend/__tests__/publicProfilePrivacy.test.js`

Verified:

- Test: 8 cases; 7 fail on the old code, all pass on the new.
- Live: another user's profile returns no private field but still the name · `/users/top` returns 0 emails (was 5 of 5) · searching by email domain finds 0 people · own profile still shows own email.

P1 H3 (part 1) · User search treats input as text

Problem: search text went straight into `new RegExp()`: `(a+)+$` could stall MongoDB and `(` returned a 500. Arrays (`?q[]=a&q[]=b`) also got through.

Fix: input must be a string, is trimmed to 100 characters and escaped with the existing `escapeRegex` in `/users/search` and `/users/search-all`. The public `/filter/apply` endpoint follows in part 2.

Verified: test cases for `(a+)+$`, `(` and array input · live: all six requests return 200.

P1 H5 · Seed scripts can no longer wipe production

Problem: all 10 seed entry points empty collections (users, jobs, orders…) and read `backend/.env`, which on the server is the production database. One mistaken run would delete every user and create `user1@jobblo.no` as superAdmin with a password written in the repo. `seeds/index.js` also printed the full `MONGO_URI`.

Fix: one shared guard (`seeds/guard.js`) runs before every `mongoose.connect` in the seed scripts:

- Refuses when `NODE_ENV=production`, even with confirmation.
- Otherwise requires the database name as confirmation: `--yes-wipe=jobblo-dev`.
- Logs only host and database name; the URI with credentials is no longer printed.
- Note: a "local database only" rule would not help here, because production's `MONGO_URI` is also `127.0.0.1`.

Files: new `backend/seeds/guard.js`; guarded `seedData.js`, `seeds/index.js`, `seeds/run-*.js` (6), `seeds/seedNorwayLocations*.js` (2); `seeds/Readme.md` updated · new test `backend/__tests__/seedGuard.test.js`

Verified: 9 tests, including starting the real `seeds/index.js`, `run-categories.js` and `seedData.js` with `NODE_ENV=production`: each exits with code 1 before connecting and never prints the password from the URI.

P1 H1 · Mobile: login links only accepted for a sign-in the app started

Problem: any app or web page can open `jobblo://oauth-success?token=…`. The app checked that the token was real and signed in as that user, even over an existing session. An attacker's link switched the phone into the attacker's account, so the victim's addresses and messages went to them.

Fix:

- `runOAuthSession` records "this app started a sign-in at <time>" in memory and storage (survives Android restarting the app).
- `completeOAuthSignIn` (the single place both iOS and Android finish a sign-in) accepts a link only within 10 minutes of that, and never while someone is signed in; the marker is cleared after a successful sign-in.
- The check runs inside the single-flight sign-in, so two deliveries of the same real link still join one request.

Files: new `jobblo-app/src/utils/oauthPending.ts` (+ `oauthPending.check.mjs`), `src/utils/oauthAuthSession.ts`, `src/hooks/useOAuthLoginCompletion.ts`

Verified: 8 checks (fresh start accepted; no start, expired, future timestamp refused; never over a signed-in account) · app typecheck 0 errors. Longer term: replace the token in the link with a one-time code exchanged with PKCE, or verified App/Universal Links.

P1 H2 · Mobile: logout switches off push notifications

Problem: `logout()` ended the server session first, then tried to deactivate the push token with that dead session. The 401 was swallowed and the local key deleted, so the device kept receiving the previous user's chat notifications.

Fix: deactivate the push token first, while the session is still valid, then end the session.

Files: `jobblo-app/src/store/authStore.ts`

Verified: replayed both orders against the real API: old order → DELETE 401 and the token stays **active**; new order → DELETE 200 and the token is **inactive**.

P1 H3 (part 2) · Unused public filter endpoint removed

Problem: `POST /api/filter/apply` was public, ran the keyword as a regular expression, took `page` / `limit` from the body without a cap (`limit: 1000000` dumped every listing), returned drafts, paid and completed jobs, dropped the location filter whenever a keyword was set, and put the raw error text in its 500 response.

Fix: neither the web app nor the mobile app calls it (only `/api/filter/options` is used), so the route, its controller function and its Swagger block were removed instead of patched.

Files: `backend/routes/filter.js` , `backend/controllers/filterController.js`

Verified: live `POST /api/filter/apply` → 404; `GET /api/filter/options` → 200; full suite green.

P1 H4 · One review per person per order

Problem: `revieweeRole` came from the request body and is part of the one-review-per-order unique index, so a party could rate the other person twice on the same order (once as `seeker` , once as `poster`).

Fix: the role is derived from the order (customer reviewing the provider → `poster` , provider reviewing the customer → `seeker` , the same mapping SafePay approval uses). A second review now hits the unique index and gets "Du har allerede gitt vurdering for dette oppdraget".

Files: `backend/controllers/reviewController.js` · test added to `backend/__tests__/reviewEligibility.test.js`

Verified: for both parties and every claimed role (`seeker` , `poster` , none) the stored role is the derived one; the new tests fail on the old code (2 of 11) and pass on the new.

Test stability · flaky queue test fixed

Problem: the "Redis down → fall back in-process" test timed the first load of the BullMQ library as well, which could exceed its limit while the full suite runs in parallel.

Fix: load BullMQ before starting the timer, so the test measures only the 3-second fallback. **Verified:** 3 full runs in a row, 1,651 passed each time.

P1 H9 · Setting up 2FA needs a code sent to the account's email

Problem: an admin or developer without an authenticator yet could set one up with only a password. Someone with a leaked password could add *their own* authenticator and then pass the 2FA gate for good.

Fix: new `POST /api/auth/mfa/setup/send-code` emails a 6-digit code to the account's address (refused when 2FA is already on; limited to 1 per minute and 5 per hour). `POST /api/auth/mfa/setup` returns the authenticator secret only after that code is checked for this user and this session, with an attempt limit. This reuses the existing developer step-up email code. The website's 2FA screen first asks "Confirm it is you" (email me a code, then continue), and only then shows the QR code.

Files: `backend/controllers/mfaController.js` · `backend/routes/auth.js` · `frontend/src/components/shared/MfaGate.tsx` · tests `backend/__tests__/mfaSetupEmailCode.test.js`, `frontend/e2e/developer-console.spec.js`

Verified: 7 new tests (all fail on the old code). Browser test: no secret is requested before the email step; a wrong code shows the error, and the right code shows the QR set-up. Live on the local backend: set-up without a code or with a wrong code gives 400 `MFA_SETUP_EMAIL_CODE` and no secret. Full backend suite: 1,658 passed.

Note: the mobile app has no 2FA set-up screen, so no app change was needed.

P1 H8 · Backend libraries updated

Problem: `npm audit` reported 8 known security bugs in backend libraries (3 high, 5 moderate). The high ones were in body-parser (request body handling), multer (file uploads) and follow-redirects (used by axios).

Fix: updates within the same major version only, so the API does not change:

Library	Before	After
body-parser	2.3.0	2.4.0
multer	2.4.0	2.5.0
follow-redirects	1.16.0	1.16.1
nodemailer	10.0.9	10.2.1
fast-uri	3.1.8	3.1.9
ip-address	10.7.3	10.7.5
uuid	13.0.0	13.0.2

nodemailer was pinned to exactly `10.0.9`, which is why `npm audit` asked for `--force`. It is now `^10.2.1`.

Files: `backend/package.json` · `backend/package-lock.json`

Verified: full backend suite, 1,658 passed. Direct checks of the new versions:

- JSON body: 200 for a valid body, 413 for one that is too large, 400 for malformed JSON.
- File upload: 200 for a normal file, 413 for one over the size limit.
- Sending email: a message with Norwegian letters is built correctly.

The local backend restarted on the new packages, and all 14 earlier live checks still pass.

Left on purpose: 2 moderate findings for `uuid@9` inside `firebase-admin → @google-cloud/storage → gaxios`. The bug only affects uuid v3, v5 and v6 when a buffer is passed, and gaxios only calls `uuid.v4()`, so it cannot be reached here. It will go away with a future firebase-admin update.

P1 H6 · Money no longer gets stuck after approval or a failed transfer

Problem: three ways a paid job ended with the money held on the platform and nothing retrying it:

1. **Approval.** The order is marked `completed` first. If a bookkeeping step after that (chat, listing, history, review, rating stats) failed, the request ended with an error before any payout existed. Approving again was refused ("already completed").
2. **Auto-release.** If the 14-day auto-release payout failed and the retry job paid it later, the payment became `released`. The auto-release job only looks at `completed` payments, so the order stayed "waiting for review" for good, and the customer could still open a dispute on money that had already been paid out.
3. **Stripe errors.** Any error from Stripe's transfer call set the payout to `reconciliation_required`, even when Stripe had clearly refused it. Nothing ever retried that state. The idempotency key was also fixed, so a retry within 24 hours would only have got Stripe's saved error back.

Fix:

1. The bookkeeping steps after approval are isolated. A failure is written to the error log (`APPROVAL_SIDE_EFFECT_FAILED`) and the payout step always runs.
2. Auto-release also accepts a `released` payment and then finishes the order. The retry job finishes the order as soon as its payout succeeds. One shared step does this (compare-and-swap, so it never touches an order a customer or dispute already moved).
3. A clear refusal from Stripe (a 4xx answer other than 409) now means "failed, no money moved": the order lock is released and the retry job tries again. Network errors and 5xx answers stay "outcome unknown".
4. Each attempt uses its own idempotency key. A second transfer is still impossible, because every retry first looks up Stripe for a transfer carrying this payout's id.
5. The retry job (every 2 hours) now also checks "outcome unknown" payouts. If the transfer did happen, it is recorded. If, 24 hours after the attempt, Stripe still has no transfer for this payout, it goes back to "failed" and is paid on the next run.

One case is left for a person on purpose: if the provider has changed payout account since the attempt, the lookup could miss the old transfer, so the payout is never retried automatically. The daily money check still reports such orders.

Files: `backend/services/payout/releasePayoutToProvider.js` · `backend/services/scheduler/jobs.js` · `backend/controllers/SafePayCheckoutController.js` · tests: `__tests__/approvalPayoutIsolation.test.js` (new), `__tests__/payoutReleaseRecovery.test.js`, `__tests__/safepayScheduler.test.js`, `__tests__/bug006_stripe_connect.test.js` (now expects one key per attempt)

Verified: 13 new tests. Each of the three paths has tests that fail on the old code (8 in total). The fake Stripe in the tests replays saved answers per key, like real Stripe. The tests show:

- A refused transfer is paid on the next retry.
- An unknown outcome is kept for 24 hours, then paid exactly once.
- A transfer that really happened is only recorded, never paid twice.
- A payout account that changed blocks the automatic retry.

Full backend suite: 1,671 passed. The local backend boots and the scheduler starts on the new code, and the retry job ran cleanly against the local database (no stuck payouts there to replay).

P1 H7 · Every dispute ends with money and order status in step

Problem: four dead ends in the admin dispute flow:

1. A reopened dispute kept its old resolution, so it could never be resolved again, and the payout stayed frozen.
2. An admin could set a dispute to `closed` or `cancelled` without resolving it, which left the order `disputed` with no way out.
3. On a paid-out order, the customer was refunded *before* the provider's transfer was reversed. If the reversal then failed (the provider had already been paid out), the platform paid the money twice.
4. A "full refund" returned only the job price, not the 3 % fee the customer had also paid.

Fix:

1. A dispute now ends only through "Resolve". The status endpoint refuses `resolved`, `closed` and `cancelled`; the admin web app never used it for those.
2. Reopening clears the old resolution and puts the order (and payment) back under dispute. It uses the same safety checks as opening a dispute: not while a payout or start-deadline refund holds the order, and not if another dispute is active.
3. A resolution that has already moved money (a refund, transfer or reversal) cannot be reopened, so money can never move twice; that needs a manual correction in Stripe. Old `cancelled` dead-end disputes can now be reopened, and the admin page shows the button for them.
4. The provider's transfer is reversed first, and the customer is refunded only after that succeeds. A failed reversal refunds nothing and leaves the dispute open to retry.

Decision made (policy): a full refund (`full_refund_to_customer` and `cancel_without_payment`) now returns everything the customer paid, **the 3 % fee included**. This is the same as the existing start-deadline refund, and it is the customer-safe choice. To keep the fee instead, change `paidTotal` back to `securedAmount` in `disputesAdminController.js`.

Files: `backend/controllers/admin/disputesAdminController.js` · `frontend/src/pages/SuperAdminDashboard/DisputeDetailPage.tsx` · test: `backend/__tests__/disputeResolutionMoney.test.js`

Verified: 11 new tests, all failing on the old code. They cover:

- A full refund includes the fee.
- The reversal runs before the refund, and a failed reversal refunds nothing.
- Status changes cannot end a dispute.
- Reopen, then resolve again, works; reopen is refused after money moved, while the order is locked, or with another active dispute.

The 16 existing dispute tests still pass.

P2 M1 · Error responses no longer leak internal messages

Problem: controllers answered 500 errors with `error.message`, which sent raw MongoDB and Stripe text to the client (collection names, ids, and e-mail addresses inside duplicate-key errors). Every unknown URL (404) was also logged as `INTERNAL_SERVER_ERROR`, and the error logs never expired.

Fix:

- One helper, `utils/serverError.js`, writes the full error to the log under the request id and answers with a safe message plus that id (`referenceId`), so support can find the log entry. It replaces 40 raw responses in 11 controllers.
- For catch-alls around a create or update, a sibling helper answers a validation error with 400 and its message (the problem is in the caller's input), a cast error with a neutral 400, and anything else with the safe 500.
- Where an error has its own `status` (a refusal our code made on purpose, such as a wrong 2FA code), its message is kept. Otherwise it goes through the helper.
- The central error handler no longer logs 4xx errors as server errors.
- A client-sent `X-Request-ID` is only echoed back when it is short and plain (at most 64 letters, digits, `-` or `_`).
- Error entries now expire after 90 days, in both `ErrorLog` and the error part of `AdminActivity`. The audit trail is kept.

Files: `utils/serverError.js` (new) · `middleware/errorHandlers.js` · `middleware/requestId.js` · `middleware/auth.js` · `models/ErrorLog.js` · `models/AdminActivity.js` · 20 controllers · test `__tests__/safeErrors.test.js`

Verified: 9 tests. They check that:

- No response contains duplicate-key or cast-error text.
- The request id is in the response and in the log.
- A 404 is not logged.
- A spoofed request id is replaced.
- A **scan of every controller** fails the build if a raw `.message` is ever sent with a 500 again.

Both TTL indexes are built on the local database, and the full backend suite passes.

P2 M12 · No personal data in logs

Problem: about 220 places logged whole error objects (a duplicate-key error contains the user's e-mail or phone number), the error log stored raw messages, the redaction list missed e-mail, phone, address, birth date and codes, a controller printed a user's full transaction list on every call, and the mobile app logged an Axios error that carries the user's token.

Fix:

- The redaction list now also covers e-mail, phone, address, birth date, national id, OTP, e-mail codes and backup codes.
- A new `redactText` function masks e-mail addresses, duplicate-key values and phone or id numbers inside free text. The error log applies it to messages and stack traces.
- At server start, every console line goes through the same redaction. That is one place instead of 220 call sites, and it also covers future ones.
- The transaction dump is removed.
- The mobile app logs only the message and HTTP status.

Files: `utils/sanitizer.js` · `utils/errorLogger.js` · `utils/consoleRedaction.js` (new) · `bootstrap/index.js` · `controllers/transactionController.js` · `jobblo-app/src/services/categories.service.ts`

Verified: tests for text and object redaction and for console output (Error objects included). The app typecheck is clean.

Also fixed: `redisManagement.test.js` failed once a local Redis held about 31,000 keys, because its bounded SCAN never reached the test's own keys. The test now uses its own Redis database (`/15`).

P2 M10 · Webhooks from both Stripe modes are processed

Problem: the webhook took its signing secret and Stripe client from the global `STRIPE_TEST_MODE` switch. After a switch, the other mode's events (live refunds, chargebacks, transfers) failed the signature check or were rejected as a "mode mismatch". Stripe stops retrying after about 3 days, so they would be lost. A paid checkout from the other mode was also sent to manual review.

Fix:

- The signature is checked against both signing secrets, test and live.
- The event's own `livemode` must match the secret that verified it.
- The event is then processed with that mode's Stripe client. If that mode has no API key, the answer is 503 so Stripe retries.
- Duplicate handling (StripeEvent) is unchanged.
- Checkout confirmation compares the session with the order's own mode, not the switch.

Files: `backend/config/stripe.js` · `backend/services/stripe/webhookDispatcher.js` · `backend/controllers/SafePayCheckoutController.js` (mode check only) · tests `stripeWebhookBothModes.test.js` (new), `confirmPaidSessionIdempotency.test.js`, `stripeWebhookDispatcher.test.js`

Verified: 6 new tests use real Stripe test signatures. With the switch on test, a live refund is processed with the live client. The old code fails that test. A forged event is refused, and so is an event signed with the wrong mode's secret. 52 tests in the three Stripe suites pass; I re-ran them myself.

Owner action: set all four on the server: `STRIPE_WEBHOOK_SECRET`, `STRIPE_TEST_WEBHOOK_SECRET`, `STRIPE_SECRET_KEY` and `STRIPE_TEST_SECRET_KEY`. If Connect events go to a separate Stripe endpoint, that endpoint has its own secret, which is not supported yet; today it must be the same endpoint as the platform one.

P2 M2 · Every list endpoint has a page limit

Problem: about 20 endpoints used `limit` / `page` exactly as sent, or returned the whole collection: favorites, lists, orders, job requests, reviews, applicants, a user's listings, notifications, transactions, blocked users, admin lists, and others. A huge, negative or text value loaded entire collections or caused 500s.

Fix: the shared `parsePagination` helper is used everywhere. It accepts whole numbers only, caps the limit at 100, and allows page 1–1000 (the page cap is new). Bad input is clamped to a safe value, not refused, so existing clients keep working. Where the web or app expects every item, the default page size is 100, and response shapes are unchanged. List endpoints that had no order are now sorted newest first, so pages are stable. The admin transaction search now treats its input as plain text, not a regex.

Files: `utils/pagination.js`, plus these controllers: favorites, lists, orders, reviews, applicants, notifications, transactions, blocking, admin and my-applications; test `listBoundsAndAuthz.test.js`

Verified: each endpoint, called with `limit=99999999&page=-5`, queries at most 100 items. The tests fail on the old code.

Not capped on purpose: the owner's own "my posted jobs" and applicants overview, where clients need the full list.

P2 M6 · Smaller authorization gaps closed

- **Time entries:** any logged-in user could read any job's time entries. Now only the job owner or the assigned worker can. One shared check (`isOwnerOrAssignedWorker`) replaces two copies.
- **Other users' listings:** `GET /users/:id/services` showed other people's drafts, cancelled and completed listings. Others now see public listings only; the owner sees everything.
- **System notifications:** marking one as read marked it read for *every* user. "Read" is now stored per user (`readBy`), and the list, the unread badge and the live (socket) count all use that. The list of who read a notification is no longer sent to clients.
- **Public config:** `GET /api/config/:key` returned any setting, including internal ones such as the Stripe mode. Only an allowlist of public keys is served now; any other key returns 404 without a database query.
- **Messages** (marking another user's message as read, sending messages typed as "system"): handled by the chat agent, see M5.

Files: `controllers/services/progressController.js` · `controllers/services/browseController.js` · `routes/users.js` · `controllers/notificationController.js` · `services/notifications/index.js` · `controllers/globalConfigController.js` · test `listBoundsAndAuthz.test.js`

Verified: 15 tests in the file (M2 and M6); 14 fail on the old code. I re-ran them together with the existing notification tests: 22 passed.

Behaviour change: unread system announcements now count towards the bell badge. Old announcements that the bug had already marked read stay read.

P2 M9 (+ P3 mobile) · Mobile app security and App Store risks

- **Tokens:** the login token now lives in the phone's secure storage (expo-secure-store) instead of plain storage. It is excluded from backups and never moves to a new device. A token saved by an older app version is moved over once, and a phone without secure storage falls back safely. Android backup is switched off.
- **Half-written job:** the draft is now stored per user and deleted on logout, on account switch and on forced logout, so the next user never sees it.
- **"Betal" double tap:** the button stays disabled until Checkout has opened. The cause was that the payment library re-enabled it too early.
- **App Store 4.8:** Google sign-in is hidden on iOS until Sign in with Apple exists.
- **App Store 5.1.1:** the unused microphone, "always" location, precise-location and motion permissions are removed.
- **App Store 3.1.1:** buying a membership is blocked on iOS whatever the configuration says.
- **Monitoring:** screen and API paths no longer contain ids, UUIDs or Stripe ids.
- **iOS push:** the backend only sends through Firebase (FCM), which cannot use an iOS (APNs) token. So instead of asking for a permission that delivers nothing, iOS push is switched off for now and the settings screen explains why.

Files: `jobblo-app/src/utils/authStorage.ts`, `draftStorage.ts`, `src/store/authStore.ts`, `app/(app)/create-job.tsx`, `app/(app)/safepay/checkout/[orderId].tsx`, `app/(auth)/login.tsx`, `src/config/features.ts`, `src/monitoring/paths.ts`, `reporter.ts`, `src/services/pushNotifications.service.ts`, the notification settings screen, `app.json`, `package.json` (expo-secure-store)

Verified: 4 checks (secure storage and migration, per-user drafts, path cleaning, OAuth link), each failing on the old code. I re-ran them all myself and they pass. The app typecheck has 0 errors.

Owner actions:

- Make a new EAS build: secure storage and the removed permissions take effect only in a new native build.
- Decide on Sign in with Apple. This brings Google sign-in back on iOS, and Vipps login may also need it under rule 4.8.
- Check whether paid extra contacts count as digital goods on iOS.
- For real iOS push, add Firebase iOS messaging (or APNs/Expo push sending on the backend).

P2 M5 · Sessions and sockets (and the chat parts of M2, M4, M6, M7 and P3)

- **Old refresh token:** the grace window used `updatedAt`, which every request changes, so an old token kept working. It now uses a separate `rotatedAt`. Reusing an old token outside the 60-second window is treated as theft: the session is deleted and its sockets are disconnected. Inside the window, the current token is returned without rotating again.
- **Sockets after logout:** every socket joins a session room. Hooks on the Session model disconnect a session's sockets whenever that session is deleted, which covers every place that ends a session:
  - logout, and revoking one or all sessions;
  - password change and reset;
  - admin deactivate and revoke;
  - self-delete, and the refresh-theft case above.

  An admin soft-delete now also deletes the user's sessions. This works across servers through the Redis adapter.
- **One message check:** the HTTP route and the socket now share one send function. It checks the text (non-empty, at most 5,000 characters), that the sender is a participant, the block list in both directions, a limit of 30 messages per minute per chat, and a chat size cap below MongoDB's 16 MB (a full chat answers 409).
- **Presence:** each user receives only their own chat partners' online status, not every online user's id.
- **Blocking now works (M4):** it stops creating a chat, sending chat and order messages, and applying to the other person's job.
- **Order messages (M6):** only the order's parties can mark a message read, and clients can no longer send "system" messages.
- **Lists (M2):** chats and messages are paged; the default page is 100.
- **Agreed price (P3):** must be a whole number of NOK from 1 to 1,000,000, and only the job owner (the payer) can set it.

Files: `models/Session.js` · `controllers/auth/loginController.js` (refresh) · `sockets/rooms.js` · `sockets/chat.socket.js` · `controllers/chatController.js` · `controllers/messageController.js` · `controllers/adminController.js` · `controllers/orderController.js` (block check on apply) · tests `tokenPurpose`, `chatSessionSecurity` (new), `chatSocketAuth`, `chatMoneyGuards`, `jobRequestContactUsage`

Verified: 20 tests fail on the old code. I re-ran all 7 chat, session and job-request suites myself: 119 passed (after the apply-block fix and one mock update).

Known limits: a session that expires through MongoDB's TTL (not a delete) does not disconnect its socket until the next check. Partners are loaded when a socket connects, so a brand-new chat shows presence after a reconnect.

P2 M7 · Abuse and cost limits

- **AI endpoints:** each user gets 20 AI calls per hour (the limit falls back to the IP address). Input over 8,000 characters is refused with 413 before OpenAI is called. A title, description or prompt that is not text gets 400 instead of crashing.
- **Client error reports** (`POST /api/errors/client`): bodies are capped at 32 kb, down from 12 MB, before the global parser runs. The endpoint has its own limit (30 per 15 minutes per IP), and stored fields are shortened and redacted.
- **Listing views:** one view per visitor per listing per 24 hours (Redis `SET NX` , with a bounded in-memory fallback), so refreshing a page can't push a listing up "Trending". The owner's own views were already not counted.

Files: `routes/ai.js` · `routes/errors.js` · `app.js` (one line) · `utils/firstView.js` (new) · `controllers/services/browseController.js` (view count only) · test `m7AbuseLimits.test.js`

Verified: 8 tests, including the 21st AI call getting 429 while another user is unaffected, a 40 kb error report refused, and three visits counted once. 6 of the original 7 fail on the old code, and I added the type check and its test. I re-ran the file myself: 8 of 8 pass.

P2 M3 · Uploads are checked properly

- **Real file type:** it is read from the file's first bytes, not the browser header. Only JPEG, PNG, WebP (and PDF for documents) are allowed. SVG is refused.
- **File names:** generated by the server, so the user's file name is never used.
- **Order of checks:** the type is checked right after upload parsing, and ownership is checked *before* anything is stored, so a refused request leaves no files behind.
- **Rate limit:** 60 uploads per 15 minutes per user.
- **Private storage:** dispute and order evidence, chat-report evidence, admin evidence and certifications can go to a private container (`AZURE_PRIVATE_CONTAINER`). Responses then carry read-only links that expire after 15 minutes. Without that setting, files go to the public container as before.

Files: `utils/azureUpload.js` · `middleware/upload.js` · upload, safepay, service and users routes · `app.js` (one line) · web `accept=` lists · app certification picker · test `uploadSecurity.test.js` (13 tests)

Owner actions:

1. Create a **private** container (e.g. `jobblo-private`) in Azure Storage.
2. Set `AZURE_PRIVATE_CONTAINER=jobblo-private` on the server.
3. Make sure the storage connection string contains `AccountName` and `AccountKey` , so the server can create the expiring links.
4. Optionally, move the old evidence files into the private container.

P2 M4 · Features that silently did not work

- **Chat-report evidence:** it was saved with an empty file URL. The files are now really uploaded, after all checks. The web dialog also called an endpoint that does not exist; it now sends one request.
- **Admin evidence upload:** it always answered "no files". It now has an upload step, at most 5 files of 5 MB.
- **Blocking a user:** it had no effect. It now stops chats, messages (web and socket) and job applications; see M5.
- **SafePay history for admins:** superAdmins got 403 because the check used role names that do not exist. Fixed.

P2 M11 · Deploy and CI

- **deploy.sh:**
  - On `main`, it deploys only after a green CI run, unless `--skip-ci-check` is passed.
  - The backend installs with `npm ci --omit=dev`.
  - The frontend is built into a new release folder, and the live site keeps serving the old build meanwhile.
  - After the API restarts, a health check runs for up to 60 seconds. Only when it passes is the site switched to the new build, in one atomic step, so there are no more 404s during deploys.
  - Any failure rolls back the code, the API and the frontend.
- **CI:**
  - It now also runs on pushes to `main`.
  - `npm audit --omit=dev --audit-level=high` runs for the backend.
  - Frontend lint and typecheck now block the build when the number of errors *grows*. Today there are 209 lint and 102 type errors, so they cannot block on any error yet.
  - A gitleaks secret scan was added.
- **Repo hygiene:** `.gitignore` now covers key files, service-account files and test output. Unused packages (`crypto`, `socket.io-client` and `body-parser` in the backend, `i` at the root) were removed. The dev Docker images run as a non-root user, and the dev frontend port is bound to 127.0.0.1.

Not tested: the release-folder swap uses a symlink, which this Windows PC cannot create. The rest of the script was dry-run with stand-in tools. **Watch the first real deploy.**

Owner actions:

- Install the `gh` CLI on the server and log in.
- Check that `HEALTH_URL`, or the default `127.0.0.1:5001`, matches the server.
- Add the `GITLEAKS_LICENSE` secret in GitHub. The old Azure secret in history (C1) will be flagged until it is rotated and allowlisted.
- Run `git rm --cached infra/terraform.tfvars`.
- Decide what to do with the unused `jobblo-argo-cd/`, `infra/` and `traefik-proxy/` folders.

Low issues, backend (11 items)

1. BankID verification can no longer reactivate an account an admin made inactive.
2. Deleting your own account needs your current password, or a login from the last 10 minutes for accounts without a password. The last superAdmin cannot delete their own account.
3. `PUT /users/:id` no longer accepts `role`. Roles change only through the admin endpoint, which checks for the last superAdmin and writes the audit log.
4. Login takes the same time whether or not the account exists, and there is a per-account lockout. In round 2 the lockout is being changed to email + IP, so attackers can't lock other people out.
5. Points shop: a balance can no longer be spent twice by two purchases at once (atomic check-and-spend).
6. Admin dispute amounts must be numbers of 0 or more with at most 2 decimals.
7. An admin cancel can no longer race a checkout that is just starting (compare-and-swap).
8. Approval no longer overwrites a refund state set at the same moment.
9. A mismatched extra-contact payment now raises a critical alert.
10. In production the API refuses to start on a public `HOST`, unless `ALLOW_PUBLIC_HOST=true`. **The server needs `HOST=127.0.0.1`.**
11. In production `SESSION_SECRET` is required; it no longer falls back to `JWT_SECRET`.

Files: the BankID, profile, login, shop, admin order and dispute controllers (amount validation only) · `SafePayCheckoutController.js` (approval filter) · `v2ExtraContacts.js` · `bootstrap/index.js` · `config/session.js` · tests `p3AccountGuards`, `p3MoneyGuards`, `p3BootGuards`, `adminOrderCancel`, `extraContacts`

Verified: each item has a test that fails on the old code. I re-ran the 8 suites myself: 79 tests passed.

R2 Second review round: findings fixed so far

Eight fresh, read-only reviewers looked at auth, money, admin authorization, input handling, the web app, the mobile app, infrastructure and backend correctness. These items are fixed and tested:

- **Server crash (high):** one socket message with no payload (`mark-as-read`) made the API process exit. The handler is fixed, and a process-wide handler now logs unhandled promise rejections instead of exiting.
- **Dispute payout with an unknown transfer outcome (high):** it could never be finished. The payout service now lets it reach the Stripe lookup. There is a test.
- **Retry job vs. an admin re-resolving a dispute (medium):** the race could pay the provider *and* refund the customer. Failed dispute payouts are now left to the admin's resolve step; only unknown-outcome ones go to the retry job, and only for the lookup.
- **Refund checked first (medium):** a refund bigger than what is left on the payment is now refused *before* the provider's transfer is reversed.
- **Failed refund (low):** after Stripe fails a refund, a new resolve starts a fresh attempt instead of replaying the failure.
- **Void payouts (low):** after a refund, the order's unpaid payout is cancelled, so it is not retried and alerted on forever.
- **Chargeback recovery (medium):** Stripe sends one event per dispute stage, and each opened its own incident, so the same chargeback could be taken back from the provider more than once. The cap now counts all incidents of the same Stripe dispute.
- **Body size (high):** one anonymous 12 MB JSON body blocked the server for about 4 seconds. The global JSON and form limit is now 256 kb.
- **Graceful shutdown (medium):** it never finished while a chat user was online, so every restart killed running jobs. Sockets are now closed first.
- **Socket rooms (medium):** the dead `join_service` event let any user create unlimited rooms. It is removed.
- The `X-Powered-By` header is removed.
- **AI input:** a non-text AI field now gets 400 instead of crashing.
- **Blocking:** a block between two users also stops applying to the other person's job.

Being fixed now by agents:

- The refresh-cookie operator injection (high): an anonymous request could delete any user's session.
- Parallel guessing of the email codes.
- Account pre-hijack through a linked Google or Vipps login.
- Per-account limits on OTP guesses.
- The lockout key, and linking a Vipps or BankID identity in a shared browser.
- The Bearer token taking priority over the cookie, and logout deactivating push tokens.
- `SameSite=Lax` cookies.
- Security headers in nginx, and an ecosystem file with `NODE_ENV` and the pm2 kill timeout.
- Metrics write volume.
- Input caps and escaping.
- 14 backend correctness bugs: scheduler batches, application restore, chat contract lock, maxApplicants, indexes, and others.

- Mobile and web review items.

R2 Security headers, NODE_ENV and graceful restarts (server config)

- **Security headers:** the new `deploy/nginx/jobblo-security-headers.conf` sets these headers on every response, so other sites can no longer load Jobblo in a hidden frame (clickjacking):
  - `X-Frame-Options DENY` and `frame-ancestors 'none'`
  - `nosniff`
  - HSTS (1 year)
  - Referrer-Policy

  It is included at server level and again in every location that sets its own header; otherwise nginx drops the server-level headers there.
- **NODE_ENV and HOST:** the new `backend/ecosystem.config.js` starts the API under pm2 with `NODE_ENV=production` and `HOST=127.0.0.1`. Every production safety switch depends on these.
- **Graceful restarts:** the same file sets `kill_timeout: 12000`, so pm2 no longer kills the API halfway through its graceful shutdown.
- **deploy.sh:** it now uses `pm2 startOrReload ecosystem.config.js`.

Verified: `bash -n deploy.sh` passes, the pm2 config loads with the expected values, and the headers file is included in all 4 places. I checked these myself. They are not tested on a real server, because nginx and pm2 aren't installed here.

Owner actions:

1. Copy the headers file to `/etc/nginx/snippets/`, then run `nginx -t` and reload.
2. Before enabling HSTS `includeSubDomains`, check that every `*.jobblo.no` host serves HTTPS.
3. Once, briefly (a short API outage): `cd backend && pm2 delete jobblo-api && pm2 start ecosystem.config.js && pm2 save`.

PERF Backend performance: toward 5,000 requests per second

Measured: local load test with autocannon (100 connections, 10 s per endpoint, 0 errors in every run). "Before" is the committed code, "after" the current code.

Endpoint	Before, 1 process (rps / CPU per request)	After, 1 process	After, 6 workers
GET /api/services (list)	93 rps / 8.7 ms	799 rps / 0.6 ms	1,060 rps
GET /api/services?category=…	114 rps / 6.7 ms	955 rps / 0.4 ms	1,094 rps
GET /api/categories	466 rps / 1.1 ms	1,241 rps / 0.4 ms	2,957 rps
GET /api/filter/options	641 rps / 1.1 ms	1,217 rps / 0.4 ms	4,139 rps
GET /api/services/:id	31 rps	620 rps	596 rps

Why not 5,000 here: about 11 agents keep this PC's CPU near 100 %, and even a bare Node "hello" server only reaches about 4,500 rps on it. CPU time per request is the reliable figure: cached public reads fell from 6.7–8.7 ms to 0.4–0.6 ms, 10–20 times less. On an idle 8-core Linux server, 6–8 workers at that cost add up to well over 5,000 rps for these reads. That is a projection; it still has to be measured on the server.

Changes:

- `cluster.js`: one worker per CPU (`WEB_CONCURRENCY=auto`, needs Redis). Sockets use WebSocket only, so no sticky sessions are needed. Slot 1 runs the scheduler and the other one-instance jobs.
- Response cache (`utils/cache.js`): responses are stored as Buffers, a 1-second in-process layer sits in front of Redis, and identical concurrent requests share one lookup. Invalidation is one atomic Redis call instead of a keyspace scan of about 155 round trips.
- Redis auto-pipelining.
- New indexes for the listing sorts and filters, and for top users, which was sorting the whole user collection in memory.
- Request metrics: an O(1) window (the old `Array.shift` cost up to 250 µs per request). Unmatched URLs are grouped, at most 1,000 buckets per minute, which closes a storage-cost attack.
- A load-test script: `backend/scripts/loadtest/run.js`.

Tests: `perfCache`, `perfClusterPlan`, `perfMiddleware` (including the cardinality cap, which fails on the old code), `perfQueries`.

On the server: `backend/ecosystem.config.js` sets `WEB_CONCURRENCY=auto` in pm2 fork mode. Do **not** use `pm2 -i`, or the scheduler stops running. `REDIS_URL` must be set.

- If the database provider limits connections, lower the pool with `?maxPoolSize=20` in `MONGO_URI` (each worker can open 100).
- Check that the new indexes get built.
- Check which database production really uses: `infra/cosmosdb.tf` defines Cosmos DB at 400 RU/s, which would cap writes far below 5,000 per second.

Next step (phase 2, after the security fixes in the same files land):

- Authenticated requests: no session write on every request; lean, field-limited user lookup.
- Listing views: count them in Redis and write them in batches, instead of one database write per view.

- Smaller list payloads (14.7 KB for 25 listings) and a cached total count.
- Run the filter aggregations in parallel, with the right index.
- Route ordering, so cache hits skip the middleware chain.

R2 Mobile app: second-round fixes

- **Login links:** the sign-in window now closes when a Google, Vipps or BankID attempt is cancelled or fails. Before, a forged login link was accepted for 10 minutes afterwards.
- **Push opt-out:** switching push off on a device now stays off after a restart.
- **Push token on logout:** the local push token is kept until the server confirms deactivation, so a failed logout can be retried.
- **PDF evidence:** it is recognised even with signed (private) links, and shown as a document instead of a broken image. Opening a document can no longer crash.
- **BankID on iOS:** the button no longer stays stuck on "Åpner BankID …".
- **Notifications:** one tapped while signed out is no longer replayed for the next user.
- **API address:** every build except development must use an `https://` API address.

Verified: all 11 app checks pass and the typecheck has 0 errors (I re-ran both myself).

Owner action: the EAS "preview" environment needs `EXPO_PUBLIC_API_URL` set to an https address.

R2 Input handling: second-round fixes

- `GET /api/users/top`: the address and city text is capped at 100 characters and 5 words, and the location queries have limits. Before, one request could create thousands of regex searches.
- **Emails:** the user's name in the password-reset email is HTML-escaped, so no injected links in a genuine Jobblo email.
- **Listing search:** the text is cut *before* escaping, so a 200-character search no longer leaves a broken regex that caused a 500.
- **Repeated query keys** (`?a=1&a=2`) no longer crash 8 endpoints with a 500. A new shared `queryString` helper is used at each site, including the admin error-log filters.
- **Admin error-log filter:** the route filter is escaped.
- **Regex sweep:** no other request-driven, unescaped regex is left in the backend.

Verified: `queryInputHardening.test.js`, 6 tests; 5 of them fail on the old code.

R2 Backend correctness: 14 bugs from the code review

1. **Abandoned-checkout cleanup** no longer brings back withdrawn or owner-declined applications. Only the ones the award itself declined are restored (new marker `autoDeclinedByOrderId`), one by one. A failure on one order no longer stops the whole run.
2. **Daily money check** looks at orders updated in the last 30 days, newest first. Before, it read the same 200 oldest orders forever.
3. **Auto-release and cleanup batches** rotate (new field `lastAutoAttemptAt`), so a few stuck orders can no longer block everyone else.
4. **After a "no action" dispute ruling**, the order gets a fresh review window and warning. Auto-release also treats a dispute closed inside the window as blocking.
5. **Cancelled contracts** no longer lock the chat. "Opprett kontrakt" and price changes work again, and a re-award keeps the negotiated price.
6. `maxApplicants` is now enforced.
7. **An admin cancelling an unpaid contract** reopens the listing and restores the auto-declined applications. This uses a shared helper (`services/order/releaseListing.js`) that the cleanup job uses too.
8. **New indexes** on applications by job owner. Before, the applicants page did up to 200 full scans.
9. **Opening a chat** counts its messages in the database instead of loading every message.
10. **Older-message pages** no longer repeat messages.
11. **A queue job** that lands late after the in-process fallback has already run is removed, so it doesn't run twice.
12. **The notification tray** no longer loads every reader of a broadcast.
13. **Editing a listing** validates the checklist like creating one does, and a single image is no longer split into characters.
14. **Accepting and declining** an application at the same moment can no longer both succeed.

Verified: 13 new tests (`listingLifecycleFixes.test.js` and one in `queue.fallback`), all failing on the old code, plus one dispute test I added for item 4. I re-ran the three suites: 49 tests passed.

Known limits (marked `ponytail:` in the code):

- Two simultaneous applications can still exceed `maxApplicants` by one.
- A queue worker that picks up the late job before it is removed can still run it twice.
- The money check covers at most 500 orders a month.

R2 Login and account security: second-round fixes

1. **High: anyone could log out any user.** A cookie written as `j:{"$ne":null}` was parsed into a database operator, so `POST /api/auth/logout` deleted a random session, or a chosen user's. Every cookie value that is not plain text is now dropped right after parsing, and the logout and refresh handlers only accept text.
2. **Email code guessing:** parallel guesses at the 2FA-setup / step-up email code all counted as one attempt. The attempt is now spent atomically before comparing; 20 parallel guesses get at most 5 checks.
3. **Account pre-hijack:** an attacker who registered your email and linked their own Google or Vipps login could still sign in after you reset the password. A password reset now removes linked logins, and the owner re-links them. The real fix is email verification at registration, still to do.
4. **Password-reset code guesses:** guessing was unlimited across new codes. There is now a per-account cap of 20 guesses per hour that a new code does not reset.
5. **Lockout:** anyone could lock any account. The lockout now counts per email + IP (10 failures), with a much higher limit per email (100 in 15 minutes), so the real owner can still log in from their own connection.
6. **Vipps and BankID linking:** a link started by one user, followed by a logout, could be completed by another person in the same browser. The callback now requires the same signed-in user and session.
7. **Token order:** the access cookie was read before the `Authorization` header, so a stale cookie in the phone app could make user B's requests run as user A. The header now wins, with the cookie as the fallback (socket too).
8. **Push on logout:** logout did not switch off the phone's push notifications on the server. Push tokens now record their login session, and logout deactivates them.
9. **SameSite:** auth cookies are `SameSite=Lax` instead of `None`, so they are no longer sent in hidden frames from other sites. One shared helper replaces four copies.

Verified: new test files `cookieInjection`, `stepUpAttempts`, `authResetGuards`, `authHeaderAndLogout` and `cookieSameSite`, plus updates to the Vipps, BankID and account suites. Each fails on the old code. Full backend suite: **1,854 passed, 0 failed**.

Known limits: push tokens registered before this change get their session id on the app's next start. If the 1-hour access token expires in the middle of a Vipps or BankID link, the link is refused and must be restarted.

R2 Access checks: second-round fixes

- **Favourites and lists:** they could store, and then reveal, draft, cancelled or finished listings. Adding a listing now needs it to be public or yours, and only public or own listings are filled in when the list is read.
- **Order messages:** they showed the other party's email before payment. They now show name and avatar only.
- **Admin panel and superAdmins:** it could create or promote superAdmins without step-up. superAdmin can no longer be assigned there; existing superAdmins can still be demoted.
- **Time entries:** a listing's time entries (hours, notes) are hidden on every read by default. Only the owner or the assigned worker can fetch them.
- **Second create-user path:** `POST /api/users` had no field allowlist or audit log, and nobody used it. It is removed; `POST /api/admin/users` remains.
- **SafePay create-session:** it checks who the customer is first, so outsiders can no longer probe whether an order exists or what state it is in.

Verified: `authzExposureReview` and `safepayCreateSessionOwnership`, 8 tests, 7 of which fail on the old code. I re-checked that the earlier fixes in the same files are intact: 53 tests passed.

P2 M8 · Web app bugs (+ P3 web and review items)

- **Logout:** every logout path now clears the data cache, including the admin and developer menus, a forced logout and the 2FA screen. The next user in the same tab no longer sees the previous user's chats, orders or subscription.
- **Double clicks:** "Opprett kontrakt" and "Start fiks ferdig-betaling" can no longer be clicked twice.
- **Profile loading:** the profile is fetched once per page load, and the page no longer remounts.
- **SafePay checklist:** the tick appears only after a successful save, and a failure shows an error.
- **Accessibility:** the contract modal opens with the keyboard (a real button, dialog role, Escape to close, focus handling), and every create-job label is linked to its field.
- `?next=` **redirect:** a new shared `safeLocalPath` check, used in 3 places, also rejects `/\` and tab or newline tricks. Share pop-ups open with `noopener,noreferrer`.
- **Deleting your account:** it asks for your current password, and explains when you need to log in again.
- **2FA screen:** it no longer hangs on "Laster …" after a failed status load; there is a "Try again" button.
- **Coins redeem:** one request only, with success and error messages.
- **Subscription success page:** it now waits for the login session to load before calling the API.
- **Web job draft:** the draft (address, phone, email, photos) is deleted on every logout and stored with its owner, so the next user in the same browser never sees it. This is the web version of the M9 mobile fix.
- **SafePay checklist:** it is announced as a real checkbox to screen readers.

Verified: `e2e/m8-web.spec.js` (10 browser tests) and `safeLocalPath.test.mjs` pass, and the typecheck is clean.

Owner action (Google Maps): the reverse geocoding calls Google's Geocoding web service from the browser, which does not accept a key restricted to your website. In Google Cloud Console, check that the browser key has website restrictions and API restrictions, and set quota and billing alerts. Better: switch to the Maps JavaScript Geocoder, which works with a restricted key.

PERF Backend performance, phase 2: the per-request hot spots

- **Logged-in requests:** every logged-in request used to write to the database. The session's "last used" time is now written at most once a minute, and the session and user are read as lean objects. The database work per request dropped from 5.4 ms to 2.5 ms of CPU, about 3 times the throughput in the A/B test. Bearer-first, the 2FA gate and the session revocation are unchanged.
- **Listing views:** they are counted in Redis (`HINCRBY`) and written to MongoDB in one batch every 15 seconds, instead of one database write per view. CPU per counted view fell from 557 µs to 16 µs. The listing detail page on 6 workers went from 596 to 1,694 requests/second (p99 470 → 178 ms), even though this PC was busier. The one-view-per-visitor rule and the owner exclusion are kept.
- **Listing list:** it is 33 % smaller (14.7 → 9.8 KB for 25 listings), because fields no list view uses are dropped. The total count is cached per filter, and the county lookup is cached for an hour.
- **Top users, categories and filter options:** lean reads, a cached total count, and the filter queries running in parallel instead of one after another.
- **pm2 cluster mode:** under `pm2 -i`, only instance 0 now runs the scheduler. Before, every instance ran it.

Verified: new tests `perfAuth`, `viewCounter`, `perfList`, `perfUsersTop` and `perfFilterOptions`, plus `slotFrom` cases in `perfClusterPlan`. Each fails on the old code. The lean outputs are identical to the old ones on all 24 dev listings. I re-ran the perf and auth suites myself: 89 tests passed.

Measurement note: during phase 2 this PC was far more loaded than in phase 1: a bare Node server dropped from about 4,500 to 879 rps. The raw rps numbers from that run are not meaningful; the CPU-per-request and payload numbers above are. Final 5,000 rps proof needs the load-test script on the real (idle) server:

```
node scripts/loadtest/run.js --url http://127.0.0.1:5001
```

Skipped on purpose:

- A Redis cache of the logged-in user: its invalidation could not be guaranteed on every role, status, 2FA and password change.
- Reordering routes: no measurable gain.

Known limits: view counts in MongoDB lag by up to 15 seconds, and counts held in memory are lost if a process crashes (when Redis is off).

Dev data note: the listing `6ada49e45851423aa9563b53` got inflated view counts from the phase-1 benchmark. Reset it by hand if that matters.

R3 Final review round: 66 findings, 64 fixed

Eight fresh reviewers re-checked every file after rounds 1 and 2: money, login, access to data, input and uploads, chat and sockets, web app, mobile app, and server/deploy. Every finding, with its fix and status, is in **Jobblo-Security-Review-Report.pdf**. In short:

- **Money:**
  - No second refund after re-resolving a dispute.
  - A won chargeback no longer blocks the payout.
  - A dispute cannot be opened on a payout whose outcome is unknown.
  - Status changes cannot undo a resolved dispute.
  - Chargeback incidents can close, including a new close-lost-chargeback action.
  - Recovery has a lock, so two admins cannot over-recover.
  - **Audit entries that were silently dropped are now saved.** 19 admin actions were missing from the allowed list.
- **Login:**
  - An attacker can no longer lock the owner out.
  - Password checks are limited, and all hashing uses the bounded queue.
  - 2FA and backup codes can only be used once, and repeated wrong codes end the session and email the owner.
  - Only the owner can change their own email.
  - Push tokens are switched off on every session end.
- **New sign-in features:**
  - **E-mail verification at sign-up**, with no email enumeration. This is the real fix for account pre-hijack.
  - **Sign in with Apple** in the app, so Google sign-in is shown on iOS again.
  - **One-time code + PKCE** for the app's Google/Vipps sign-in link, so the link no longer carries a token.
- **Sockets:**
  - Per-socket event limit, and at most 10 sockets per user.
  - Sockets close at token expiry and on revocation, even when Redis is down.
  - Account status is checked on connect, and the block list applies to every contract step and to presence.
  - Chats and messages are hidden instead of deleted, so evidence survives.
- **Input:**
  - The log-masking regex can no longer freeze the server (84 s became 1 ms).
  - Upload forms have field limits; one request had used 1.2 GB.
  - Cache keys ignore the query string where it doesn't matter, and image decoding has a pixel limit.
  - The remaining page caps are in place, the error log no longer stores query strings, and the AI image type is checked from the file bytes.
  - Client error reports cannot pose as server errors.
- **Access:**
  - List search shows only public listings.
  - List contributors cannot read other people's emails.

  - Image links must come from Jobblo storage, and private evidence links cannot be re-published.
  - The profile website must be a real http(s) address.
- **Mobile:**
  - Login tokens are no longer written to the nginx log.
  - A token refresh cannot switch to another account.
  - Notifications are cleared from the tray on logout.
  - Unused Android permissions are blocked.
  - Account deletion asks for the password.
- **Server, deploy and CI:**
  - Deploy always rolls back on failure and needs green CI on every branch.
  - Support tickets are limited.
  - The scheduler stops first on shutdown, and pm2 gets 16 s.
  - Login limiters fail closed when Redis is down.
  - Report-only CSP and a Permissions-Policy header.
  - Audits for the frontend and the app, and a production Docker image.
- **Web:**
  - No private chat text in monitoring.
  - The unused success page is removed.
  - The 6 bugs found by the new E2E tests are fixed.
  - Type errors went from 102 to 0, so the typecheck now blocks CI, and lint errors from 209 to 56.

Not fixable in code: C1, the Azure secret in git history, which the owner must rotate. P7, the npm audit warnings in the app's build tools, waits for the next Expo update; forcing it would break the app.

Release order: deploy the backend first, then the new app build. The new app expects the one-time sign-in code and e-mail verification.

Verified:

- **Backend:** 2,025 tests passed on a clean checkout of the commits. The only failing test checks frontend files committed in the frontend commit.
- **Mobile:** all checks and the typecheck pass.
- **Web:** the E2E suite is re-run before the frontend commit.

Generated from `docs/security/fixes.html`. Tests: backend jest suite 1,854 passed, 0 failed (latest full run).